UNIT-IV
Arrays,Strings and Pointers

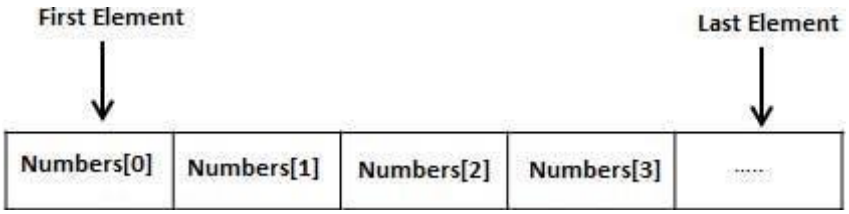
Defination of Arrays:

Array is collection of similar data elements.

Arrays a kind of data structure that can store a fixed-size sequential collection of elements of the same type. An array is used to store a collection of data, but it is often more useful to think of an array as a collection of variables of the same type.

Instead of declaring individual variables, such as number0, number1, ..., and number99, you declare one array variable such as numbers and use numbers[0], numbers[1], and ..., numbers[99] to represent individual variables. A specific element in an array is accessed by an index.

All arrays consist of contiguous memory locations. The lowest address corresponds to the first element and the highest address to the last element.



Declaring Arrays

To declare an array in C, a programmer specifies the type of the elements and the number of elements required by an array as follows —

type arrayName [arraySize];

This is called a *single-dimensional* array. The **arraySize** must be an integer constant greater than zero and **type** can be any valid C data type. For example, to declare a 10-element array called **balance** of type double, use this statement —

double balance[10];

Here *balance* is a variable array which is sufficient to hold up to 10 double numbers.

Initializing Arrays

You can initialize an array in C either one by one or using a single statement as follows —

double balance[5] = {1000.0, 2.0, 3.4, 7.0, 50.0};

The number of values between braces { } cannot be larger than the number of elements that we declare for the array between square brackets [ ].

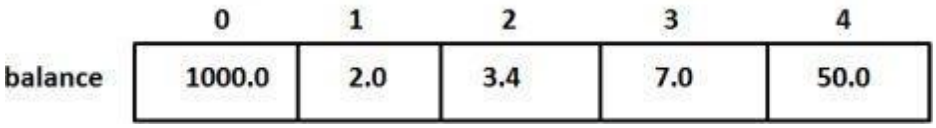
If you omit the size of the array, an array just big enough to hold the initialization is created. Therefore, if you write —

double balance[] = {1000.0, 2.0, 3.4, 7.0, 50.0};

You will create exactly the same array as you did in the previous example. Following is an example to assign a single element of the array —

balance[4] = 50.0;

The above statement assigns the 5th element in the array with a value of 50.0. All arrays have 0 as the index of their first element which is also called the base index and the last index of an array will be total size of the array minus 1. Shown below is the pictorial representation of the array we discussed above —



Accessing Array Elements

An element is accessed by indexing the array name. This is done by placing the index of the element within square brackets after the name of the array. For example —

double salary = balance[9];

The above statement will take the 10th element from the array and assign the value to salary variable. The following example Shows how to use all the three above mentioned concepts viz. declaration, assignment, and accessing arrays —

```
#include <stdio.h>

int main () {

   int n[ 10 ]; /* n is an array of 10 integers */
   int i,j;

   /* initialize elements of array n to 0 */
   for ( i = 0; i < 10; i++ ) {
      n[ i ] = i + 100; /* set element at location i to i + 100 */
   }

   /* output each array element's value */
   for (j = 0; j < 10; j++ ) {
      printf("Element[%d] = %d\n", j, n[j] );
   }

   return 0;
}
```

When the above code is compiled and executed, it produces the following result −

Element[0] = 100
Element[1] = 101
Element[2] = 102
Element[3] = 103
Element[4] = 104

Element[5] = 105
Element[6] = 106
Element[7] = 107
Element[8] = 108
Element[9] = 109

Arrays in Detail
Arrays are important to C and should need a lot more attention. The following important concepts related to array should be clear to a C programmer —

Sr.No.	Concept & Description
1	Multi-dimensional arrays C supports multidimensional arrays. The simplest form of the multidimensional array is the two-dimensional array.
2	Passing arrays to functions You can pass to the function a Pointer to an array by specifying the array's name without an index.
3	Return array from a function C allows a function to return an array.
4	Pointer to an array You can generate a Pointer to the first element of an array by simply specifying the array name, without any index.

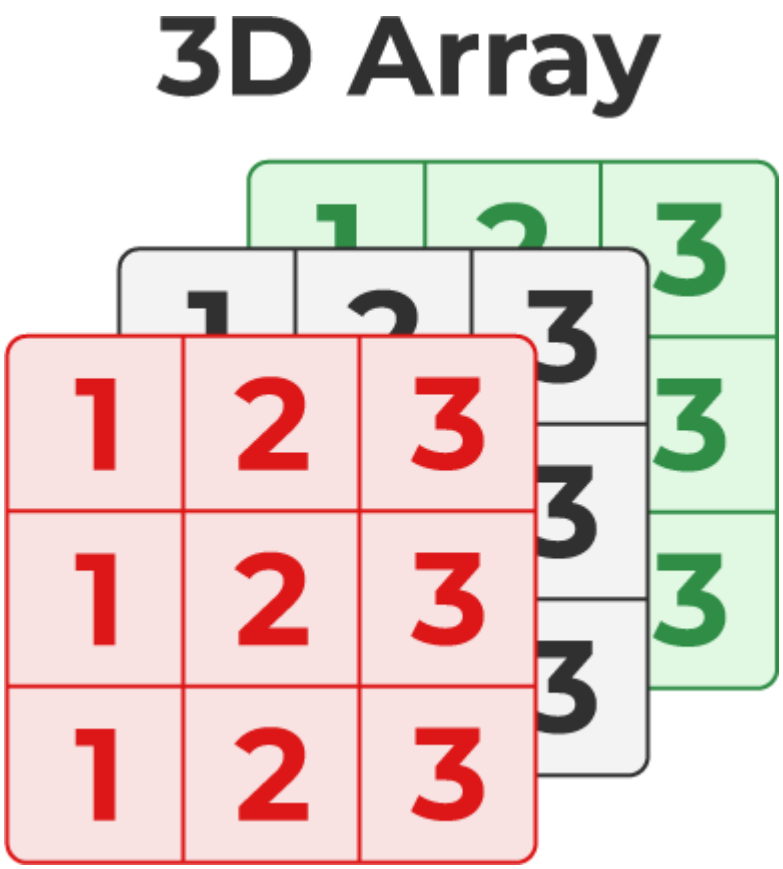
Types of Arrays:
2D Array:
10 20 30
40 50 60

B. Three-Dimensional Array in C

Another popular form of a multi-dimensional array is Three Dimensional Array or 3D Array. A 3D array has exactly three dimensions. It can be visualized as a collection of 2D arrays stacked on top of each other to create the third dimension.

Syntax of 3D Array in C

array_name [size1] [size2] [size3];



Example of 3D Array

```
// C Program to illustrate the 3d array #include
<stdio.h>
int main()

{
    // 3D array declaration
    int arr[2][2][2] = { 10, 20, 30, 40, 50, 60 };
    // print ing elements
    for(int i = 0; i < 2; i++)
    {
        for(int j = 0; j < 2; j++)
        {
            for(int k = 0; k < 2; k++)
            {
            printf("%d ", arr[i][j][k]);
            }
            printf("\n");
        }

        printf("\n \n");
    }
    return0;
}
```

Output

10 20
30 40
50 60
0 0

Processing an Array:

Processing the Arrays — For certain applications, the assignment of initial values to elements of an array is required. This means that the array is defined globally (extern) or locally as a static array.

Let us now see in the following example how the marks in two subjects, stored in two different arrays(Processing the Arrays), can be added to give another array and display the average marks in the below example.

C program to display the average marks of each student, given the marks in 2 subjects for 3 students.

```
/* Program to display the average marks of 3 students */

# include <
stdio.h > #
define SIZE 3
main()
{
int  i = 0;
float  stud_marks1[SIZE];  /*  subject  1array
declaration  */  float  stud_marks2[SIZE];
/*subject  2  array  declaration  */  float
total_marks[SIZE];
float avg[SIZE];
printf("\n Enter the marks in subject-1 out of 50 marks: \n");
for( i = 0;i<SIZE;i++)
{
printf("Student no. =%d",i+1);
printf(" Enter the marks= ");
scanf("%f",&amp;stud_marks1[i]);
}
printf("\n Enter the marks in subject-2 out of 50 marks \n");
for(i=0;i<SIZE;i++)
{
printf("Student no. =%d",i+1);
printf(" Please enter the marks= ");
scanf("%f",&amp;stud_marks2[i]);
}
for(i=0;i<SIZE;i++)
{
total_marks[i]=stud_marks1[i]+ stud_marks2[i];
avg[i]=total_marks[i]/2;
printf("Student no.=%d, Average= %f\n",i+1, avg[i]);
}
}
```

OUTPUT

```
Enter the marks in subject-1out of 50 marks:
Student  no.  =  1  Enter  the  marks=  23
Student  no.  =  2  Enter  the  marks=  35
Student no. = 3 Enter the marks= 42
Enter the marks in subject-2 out of 50 marks:
Student  no.  =  1  Enter  the  marks=  31
```

Student no. = 2 Enter the marks= 35
Student no. = 3 Enter the marks= 40
Student no. = 1 Average= 27.000000
Student no. = 2 Average= 35.000000
Student no. = 3 Average= 41.000000

Passing an array to function:

As we know that the array_name contains the address of the first element. Here, we must notice that we need to pass only the name of the array in the function which is int ended to accept an array. The array defined as the formal parameter will automatically refer to the array specified by the array name defined as an actual parameter.

Consider the following syntax to pass an array to the function.

1. functionname(arrayname);//passing array

Methods to declare a function that receives an array as an argument

There are 3 ways to declare the function which is int ended to receive an array as an argument.

First way:

1. return_type function(type arrayname[])

Declaring blank subscript notation [] is the widely used technique.

Second way:

1. return_type function(type arrayname[SIZE])

Optionally, we can define size in subscript notation [].

Third way:

1. return_type function(type *arrayname)

You can also use the concept of a Pointer. In Pointer chapter, we will learn about it.

C language passing an array to function example

```
1. #include<stdio.h>
2. int minarray(int arr[],int size)
3. {
4. int min=arr[0];
5. int i=0;
6. for(i=1;i<size;i++)
7. {
8. if(min>arr[i]){
9. min=arr[i];
10. }
11. }//end of for
12. return min;
13. }//end of function
14. int main()
15. {
16. int i=0,min=0;
17. int numbers[]={4,5,7,3,8,9};//declaration of
array 16.
17. min=minarray(numbers,6);//passing array with size
18. printf("minimum number is %d \n",min);
19. return 0;
20. }
```

Output

```
minimum number is 3
```

C function to sort the array

```
#include<stdio.h>
void Bubble_Sort(int []);
void main ()
{
   int  arr[10] = { 10, 9, 7, 101, 23, 44, 12, 78, 34, 23};
   Bubble_Sort(arr);
}
void Bubble_Sort(int  a[]) //array a[] point s to arr.
{
int  i, j,temp;
   for(i = 0; i<10; i++)
   {
      for(j = i+1; j<10; j++)
      {
         if(a[j] < a[i])
         {
            temp = a[i];
            a[i] = a[j];
            a[j] = temp;
         }
      }
   }

   printf("Print ing Sorted Element List ...\n");
   for(i = 0; i<10; i++)
   {
      printf("%d\n",a[i]);
   }
}
```

Output

```
Printing Sorted Element List ...
7
9
10
12
23
23
34
44
78
101
```

Returning array from the function

As we know that, a function can not return more than one value. However, if we try to write the return statement as return a, b, c; to return three values (a,b,c), the function will return the last mentioned value which is c in our case. In some problems, we may need to return multiple values from a function. In such cases, an array is returned from the function.

Returning an array is similar to passing the array int o the function. The name of the array is returned from the function. To make a function returning an array, the following syntax is used.

```
int  * Function_name()
{
//some statements;
return array_type;
}
```

To store the array returned from the function, we can define a Pointer which point s to that array. We can traverse the array by increasing that Pointer since Pointer initially point s to the base address of the array. Consider the following example that contains a function returning the sorted array.

```
#include<stdio.h>
int * Bubble_Sort(int []);

void main ()
{
    int  arr[10] = { 10, 9, 7, 101, 23, 44, 12, 78, 34, 23};
    int  *p = Bubble_Sort(arr), i;
    printf("print ing sorted elements ...\n");
    for(i=0;i<10;i++)
    {
        printf("%d\n",*(p+i));
    }
}
int * Bubble_Sort(int  a[]) //array a[] point s to arr.
{
int  i, j,temp;
    for(i = 0; i<10; i++)
    {
        for(j = i+1; j<10; j++)
        {
            if(a[j] < a[i])
            {
                temp = a[i];
                a[i] = a[j];
                a[j] = temp;
            }
        }
    }
    return a;
}
```

Output

```
Printing Sorted Element List ...
7
9
10
12
23
23
34
44
78
101
```

Array of Strings:

In C programming String is a 1-D array of characters and is defined as an array of characters. But an array of strings in C is a two-dimensional array of character types. Each String is terminated with a null character (\0). It is an application of a 2d array.

Syntax:

char variable_name[r] = {list of string};

Here,

- **var_name** is the name of the variable in C.
- **r** is the maximum number of string values that can be stored in a string array.
- **c** is a maximum number of character values that can be stored in each string array.

Example:

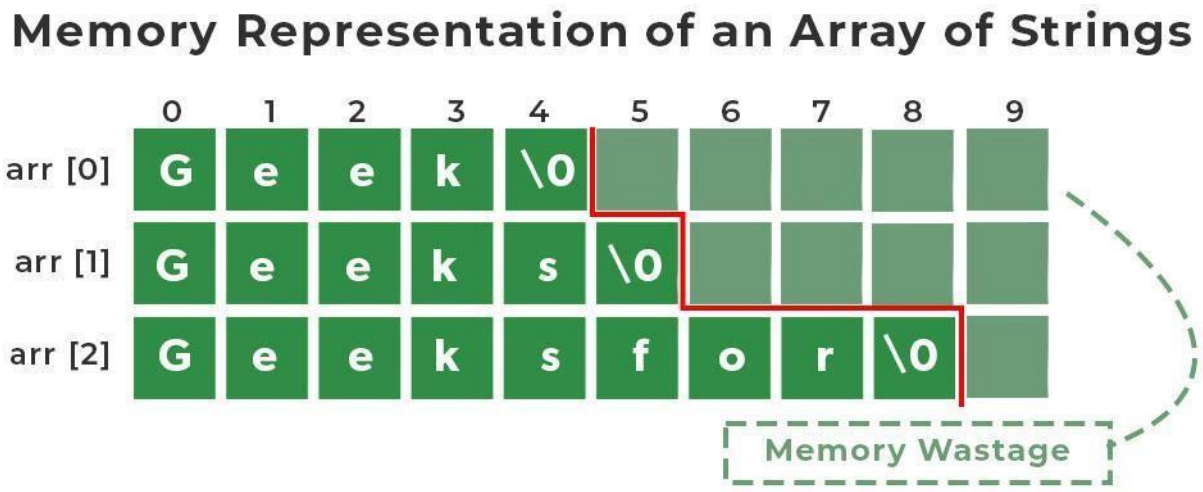
- C

```
// C Program to print  Array

// of strings #include
<stdio.h>

// Driver code int
main()
{
    chararr[3][10] = {"Geek","Geeks", "Geekfor"}; printf("String array Elements are:\n");
    for(int i = 0; i < 3; i++)
    {
        printf("%s\n", arr[i]);
    }
    return0;
}
```

Output

String array Elements are:
Geek Geeks
Geekfor

Below is the Representation of the above program



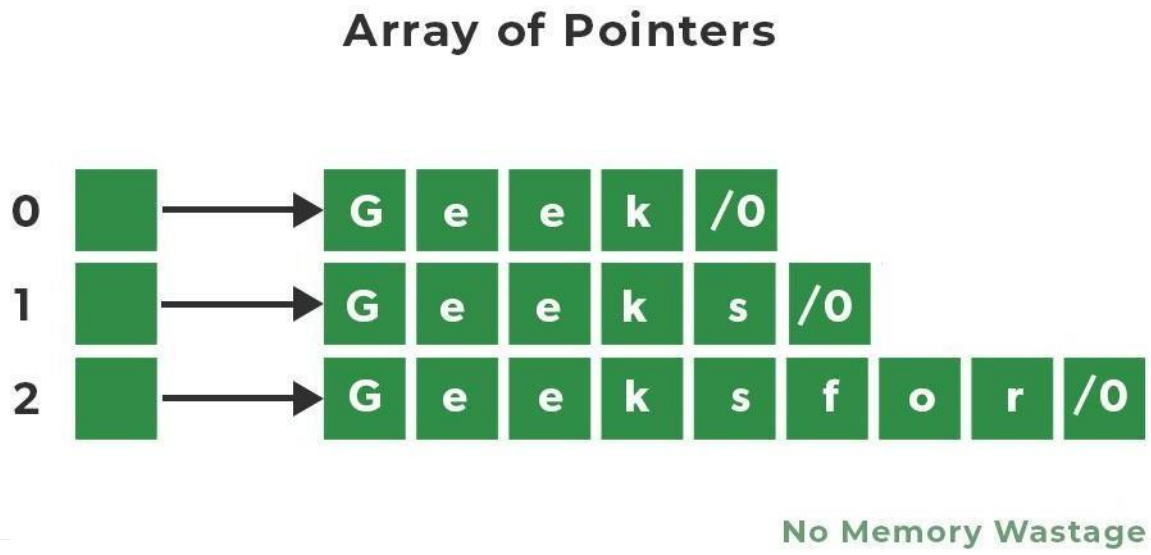
We have 3 rows and 10 columns specified in our Array of String but because of prespecifying, the size of the array of strings the space consumption is high. So, to avoid high space consumption in our program we can use an Array of Pointers in C.

Invalid Operations in Arrays of Strings

We can't directly change or assign the values to an array of strings in C.
Example:
char arr[3][10] = {"Geek", "Geeks", "Geekfor"};
Here, arr[0] = "GFG"; // This will give an Error which says assignment to expression with an array type.
To change values we can use strcpy() function in C
strcpy(arr[0],"GFG"); // This will copy the value to the arr[0].

Array of Pointers of Strings

In C we can use an Array of Pointers. Instead of having a 2-Dimensional character array, we can have a single-dimensional array of Pointers. Here Pointer to the first character of the string literal is stored.
Syntax:
char *arr[] = { "Geek", "Geeks", "Geekfor" };



Below is the C program to print an array of Pointers:

```
// C Program to print  Array

// of Pointers #include
<stdio.h>
// Driver code int
main()
{
    char*arr[] = {"Geek", "Geeks", "Geekfor"}; printf("String array
    Elements are:\n");
    for(int i = 0; i < 3; i++)
    {
        printf("%s\n", arr[i]);
    }
    return0;
}
```

Output
String array Elements are:
Geek Geeks
Geekfor

String Constant and String Variable: String Constant:

- A character string, a string constant consists of a sequence of characters enclosed in double quotes.
- A string constant may consist of any combination of digits, letters, escaped sequences and spaces. Note that a character constant 'A'

and the corresponding single character string constant "A" are not equivalent.

'A' - Character constant - 'A'
"A" - String Constant - 'A' and 0\ (**NULL**)

The string constant "A" consists of character **A** and **\0.** However, a single character string constant does not have an equivalent integer value. It occupies two bytes, one for the ASCII code of **A** and another for the NULL character with a value **0**, which is used to terminate all strings.
Valid String Constants**: -** "W" "100" "24, Kaja Street"

Invalid String Constants**: -** "W the closing double quotes missing
Raja" the beginning double quotes missing

Rules for Constructing String constants

1) A string constant may consist of any combination of digits, letters, escaped sequences and spaces enclosed in double quotes.
2) Every string constant ends up with a **NULL** character which is automatically assigned (before the closing double quotation mark) by the compiler.

String Variable:

Strings are useful for communicating information from the program to the program user and, thus are a part of all programming languages and applications. They are implemented with characters arrays that behave like usual arrays with which we can perform regular operations, including modification. Another way of implementing strings is through Pointers since character arrays act as Pointers. These Pointers point to string literals and cannot be modified as string literals are stored in the read-only memory by most compilers. Thus, any attempt at modifying them leads to undefined behavior.

What is a String?
A string is a collection of characters (i.e., letters, numerals, symbols, and punctuation marks) in a linear sequence. In C, a string is a sequence of characters concluded with a NULL character '\0'. For example:

```
char str[] = "Scaler.\0";
```

Like many other programming languages, strings in C are enclosed within ***double quotes(" ")***, whereas characters are enclosed within ***single quotes(' ')***. When the compiler finds a sequence of characters enclosed within the double quotation marks, it adds a null character (***\0***) at the end by default.

Thus, this is how the string is stored:

A string literal is a sequence of zero or more multibyte characters enclosed in double quotes, as in "abc". String literals are not modifiable (and are placed in read-only memory). Any attempt to alter their values results in undefined behavior.

Declaring a String in C
A String in C is an array with character as a data type. C does not directly support string as a data type, as seen in other programming languages like C++. Hence, character arrays must be used to display a string in C. The general syntax of declaring a string in C is as follows:

```
char variable[array_size];
```

Thus, the classic declaration can be made as follows:

```
char str[5];
char str2[50];
```

It is vital to note that we should always account for an extra space used by the null character(\0).

Highlights:

1. Character arrays are used for declaring strings in C.
2. The general syntax for declaring them is:

```
char variable[array_size];
```


Initializing a String in C
There are four methods of initializing a string in C:

1. Assigning a string literal with size

We can directly assign a string literal to a character array keeping in mind to keep the array size at least one more than the length of the string literal that will be assigned to it.
Note While setting the initial size, we should always account for one extra space used by the null character. If we want to store a string of size **n**, we should set the initial size to be **n+1**.

For example:

```
char str[8] = "Scaler.";
```

The string length here is 7, but we have kept the size to be 8 to account for the Null character. **The compiler adds the Null character(\0) at the end automatically**.
Note: If the array cannot accommodate the entire string, it only takes characters based on its space. For example:

```
char str[3] = "Scaler.";
printf("%s",str);
```

Output:

```
Sca
```

2. Assigning a string literal without size

It is also possible to directly assign a string literal to a character array without any size. The size gets determined automatically by the compiler at compile time.

```
char str[] = "Scaler.";
```

The critical thing to remember is that the name of the string, here "*str*" acts as a Pointer because it is an array.

3. Assigning character by character with size

We can also assign a string character by character. However, it is essential to set the end character as '\0'. For example:

```
char str[8] = {'S', 'c', 'a', 'l', 'e', 'r', '.','\0'};
```

4. Assigning character by character without size

Like assigning directly without size, we also assign character by character with the Null Character at the end. The compiler will determine the size of the string automatically.

```
char str[] = {'S', 'c', 'a', 'l', 'e', 'r', '.','\0'};
```

Highlights: There are four methods of initializing a string in C:

1. Assigning a string literal with size.
2. Assigning a string literal without size.
3. Assigning character by character with size.
4. Assigning character by character without size.

Assign value to strings

Character arrays cannot be assigned a string literal with the '=' operator once they have been declared.

```
char str[100];
str = "String.";
```

This will cause a compilation error since the assignment operations are not supported once strings are declared. *To overcome this, we can use the following two methods:*

1. Assign the value to a character array while initializing it, explained above.
2. We can use the strcpy() function to copy the value we want to assign to the character array. The syntax for **strcpy()** is as follows:

```
strcpy(char* destination, const char* source);
```

It copies the string point ed by the source (including the null character) to the destination. For example:

```
char str[20];
strcpy(str,"Strings.");
```

This assigns the value to the string.

Note - It is vital to make sure the value assigned should have a length less than or equal to the maximum size of the character array.

Highlights:

1. Character arrays cannot be assigned a string literal with the '=' operator once they have been declared.
2. Assignment can either be done at the initialization time or by using the strcpy() function.

Read String from the user
The most common operation used for reading a string is scanf(), which reads a sequence of characters until it encounters whitespace (i.e., space, newline, tab, etc.). The following section explains the method of taking input with whitespace. For example,

```
char str[25];
```

```
scanf("%s", str);
```

If we provide the following **input**:

```
Scaler is amazing.
```

We get the following **Output**:

```
Scaler
```

As we can see, scanf() stops taking input once it encounters whitespace.
Note -

1. The format specifier used to input and output strings in C is *%s*.
2. You might notice that generally, the variable's name is preceded by a *&* operator with *scanf()*. This is not the case here, as the character array is a Pointer that point s to the address of the first character of the array. Hence the address operator (*&*) doesn't need to be used.

Highlights:

1. Strings in C can be read using *scanf()*. However, it only reads until it encounters whitespace.

How to read a line of text?
The scanf() operation cannot read strings with spaces as it automatically stops reading when it encounters whitespace. To read and print strings with whitespace, we can use the combination of fgets() and puts():

- ***fgets()*** The *fgets()* function is used to read a specified number of characters. Its declaration looks as follows:

```
fgets(name_of_string, number_of_characters, stdin);
```

name_of_string*: It is the variable in which the string is going to be stored. **number_of_characters**: The maximum length of the string should be read. **stdin**: It is the filehandle from where the string is to be read.

- ***puts()*** *puts()* is very convenient for displaying strings.

```
puts(name_of_string);
```

name_of_string: It is the variable in which the string will be stored.

An example using both functions:

```
#include <stdlib.h>
#include <stdio.h>

int main() {
```

```
  char str[30];
  printf("Enter string: ");
  fgets(str, sizeof(str), stdin);
  printf("The string is: ");
  puts(str);

  return 0;
}
```

Input:

```
Enter string: Scaler is amazing.
```

Output:

```
The string is: Scaler is amazing.
```

In the above example, we can see that the entire string with the whitespace was stored and displayed, thus showing us the power of *fgets()* and *puts()*.

Highlights:

1. The combination of *fgets()* and *puts()* is used to tackle the problem of reading a line of text with whitespace.
2. **Syntax for *fgets()*:**

```
fgets(name_of_string, number_of_characters, stdin);
```

3. **Syntax for *puts()*:**

```
puts(name_of_string);
```

Passing strings to functions
As strings are simply character arrays, we can pass strings to function in the same way we pass an array to a function, either as an array or as a Pointer. Let's understand this with the following program:

```c
#include <stdio.h>

void pointer(char * str) {
  printf("The string is : ");
  puts(str);
  printf("\n");
}

void array(char str[]) {
  printf("The string is : ");
  puts(str);
  printf("\n");
}

int main() {

  char str[25] = "Scaler is amazing.";

  pointer(str);
  array(str);
  return 0;
}
```

Output:

```
The string is : Scaler is amazing.

The string is : Scaler is amazing
```

As we can see, both of them yield the same Output.

Highlights:

1. String can be passed to functions as a character array or even in the form of a Pointer.

Strings and Pointers

As we have seen, strings in C are represented by character arrays which act as Pointers. Hence, we can use Pointers to manipulate or even perform operations on the string.

```c
#include <stdlib.h>
#include <stdio.h>

int main() {
    char str[] = "Scaler.";
    printf("%c", *str); // Output: S
    printf("%c", *(str + 1)); // Output: c
    printf("%c\n", *(str + 6)); // Output: .

    char * stringPtr;
    stringPtr = str;
    printf("%c", *stringPtr); // Output: S
    printf("%c", *(stringPtr + 1)); // Output: c
    printf("%c", *(stringPtr + 6)); // Output: .

    return 0;
}
```

Output:

```
Sc.
Sc.
```

Note: Since character arrays act like Pointers, we can easily use Pointers to manipulate strings.

String Example in C
Here is a program that demonstrates everything we have learned in this article:

```
#include <stdio.h>

void array(char str[]) {

  printf("This function handles string literals with character arrays.\n");

  printf("First character : %c\n", str[0]);

  printf("The entire string is : %s\n", str);
  str[0] = 'Q'; //Here we have assigned the first element of the array to
Q.

  printf("The new string is : %s\n", str);
}

void literal(char * str) {

  printf("This function handles string literals using pointers.\n");

  printf("First character : %c\n", str[0]);

  printf("The entire string is : %s\n", str);
  // str[0] = 'Q';
  //Modification is not possible with string literals, since they are
stored on the read-only memory.
}

int main() {
  char str[] = "Strings."; //Here we have assigned the string literal to a
character array.
  array(str);
  printf("\n");

  char * strPtr = "Strings."; ////Here we have assigned the string literal
to a pointer.
  literal(strPtr);

  return 0;
}
```

Output:

```
This function handles string literals with character arrays.
First character : S
The entire string is : Strings.
The new string is : Qtrings.

This function handles string literals using pointers.
First character : S
The entire string is : Strings.
```

Difference between character Array and String literal
Before actually jumping to the difference, let us first recap string literals. A *string literal* is a sequence of zero or more multibyte characters enclosed in double quotes, as in "xyz". String literals are not modifiable (and are placed in read-only memory). An attempt to alter their values results in undefined behavior.

String literals can be used to initialize arrays. We have seen multiple examples of this throughout the article. Now, let us see the differences using the following examples:

```
1. char str[] = "Scaler.";
2.
```

This statement creates a character array that has been assigned a string literal. It behaves as a usual array with which we can perform regular operations, including modification. The only thing to remember is that although we have initialized 7 elements, its size is 8, as the compiler adds the *\0* at the end.

```
2. char *str   = "Scaler.";
3.
```

The above statement creates a Pointer that point s to the string literal, i.e. to the first character of the string literal. Since we now know that string literals are stored in a read-only memory location, thus alteration is not allowed.
Note - While print ing string literals, it is defined to declare them as constants like:

```
const char *str = "Strings.";
```

This prevents the warning we get while using *printf()* with string literals.

Declaration and Initialization of String:
A C String is a simple array with char as a data type. 'C' language does not directly support string as a data type. Hence, to display a String in C, you need to make use of a character array.
The general syntax for declaring a variable as a String in C is as follows,
char string_variable_name [array_size];
The classic Declaration of strings can be done as follow:
char string_name[string_length] = "string";
The size of an array must be defined while declaring a C String variable because it is used to calculate how many characters are going to be stored inside the string variable in C. Some valid examples of string declaration are as follows,
char first_name[15]; //declaration of a string variable char last_name[15];
The above example represents string variables with an array size of 15. This means that the given C string array is capable of holding 15 characters at most. The indexing of array begins from 0 hence it will store characters from a 0-14 position. **The C compiler automatically adds a NULL character '\0' to the character array created.**

How to Initialize a String in C?
Let's study the String initialization in C. Following example demonstrates the initialization of Strings in C,
char first_name[15] = "ANTHONY";
char first_name[15] = {'A','N','T','H','O','N','Y','\0'}; // NULL character '\0' is required at end in this declaration
char string1 [6] = "hello";/* string size = 'h'+'e'+'l'+'l'+'o'+"NULL" = 6
*/
char string2 [ ] = "world"; /* string size = 'w'+'o'+'r'+'l'+'d'+"NULL" = 6 */

char string3[6] = {'h', 'e', 'l', 'l', 'o', '\0'} ; /*Declaration as set of characters ,Size 6*/
In string3, the NULL character must be added explicitly, and the characters are enclosed in single quotation marks.
'C' also allows us to initialize a string variable without defining the size of the character array. It can be done in the following way,
char first_name[ ] = "NATHAN";
The name of Strings in C acts as a Pointer because it is basically an array.

Input/Output of String Data:

What is the Input and Output of a String in C?

Taking string input in C means asking the user to provide some information for the program to manipulate and analyse and storing this information in the form of a string. Similarly, giving output as string means print ing int o the console some information which the programmer wants the user to know in the form of strings. It is necessary to use strings as input and output strings to the user in C as shown in the example above.

While taking string input in C, we basically have two scenarios: strings that have spaces and strings without spaces. In the next sections, we will look at both these cases and the functions that can be used for the same.

How to Take Input of a String in C Without Spaces?

In C, we can use scanf() to take a string input in C without spaces. Like other data types, we have an access specifier (also known as format specifier) to take input as a string in C as well. The access specifier for string is %s.

The syntax for using scanf()function in C :

```
scanf("%s", char *s);
```

Here, s is the Pointer which point s to the array of characters where the input taken as a string will be stored.
Note that in the syntax, we don't use the & symbol with s. This is because & is used to access the address of the variable, but as C doesn't have a string data type, s already stores the address of the character array where the string is stored.

Let us look at an example to understand the working of scanf() in C.

```
#include <stdio.h>

int main()
{
    // array to store string taken as input
    char color[20];

    // take user input
    printf("Enter your favourite color: ");
    scanf("%s", color);

    // printing the input value
    printf("Your favourite color is: %s.", color);

    return 0;
}
```

Output

```
Enter any sentence: Orange
Your favourite color is: Orange.
```

Advantages and Disadvantages of using scanf()

The scanf() function is probably the easiest way to string input in C. We just need the access specifier %s and the Pointer to the variable where we want the string to be stored, and it becomes very easy to input the string in C.

On the other hand, there is no limitation on the size of the string that can be taken as an input by scanf() by default. This may cause overflow if we don't have the required space. But this can be easily fixed by specifying an explicit upper bound of memory which can prevent overflow. Apart from this, there is a chance of a failed input which would lead to our variable Pointer point ing to an unknown location, which can lead to further problems in the code.

Hence, the scanf() function should be preferred when we know the string we want to input doesn't contain white spaces and that it will not exceed the memory buffer.
We will explore the output of strings without space in C in the later sections of the article. Let us first see how to take input for string in C with spaces.

How to Take Input of a String With Spaces in C?

In C, there is a problem with simply taking the string as input using access specifier %s. If we take any string as input in C which has a whitespace, i.e., space, tab or newline character in it, then only the part before the whitespace would be taken as the input. Let's take a look at an example to understand this issue better.

```c
#include <stdio.h>

int main()
{
    // array to store string taken as input
    char sentence[20];

    // take user input
    printf("Enter any sentence: ");
    scanf("%s", sentence);

    // printing the input value
    printf("You entered: %s.", sentence);

    return 0;
}
```

Output

```
Enter any sentence: This is my sentence
You entered: This.
```

Even though we entered This is my sentence, the char array sentence only stored This and the remaining part was discarded because the %s access specifier works in a way that it reads the sequence of characters until it encounters whitespace (space, newline, tab, etc). Thus, as the first space is encountered just after the word This the char array sentence stores only the string This. Hence, it is impossible to take input of a string with spaces by just using the %s access specifier in scanf() function. Then how do we take such strings as an input in C?

Let's take a look at some methods which we can use to take a string input in C with spaces.

Methods to Accept String With Space in C
We'll look at four different methods to take a string input in C with spaces(whitespaces).

1. gets()

This is a C standard library function. gets() takes a complete line as input and stores it in the string provided as a parameter. The working of gets() is such that it keeps reading the input stream until it encounters a newline character: \n. Hence, even if the string input in C has spaces in it, all of them get included as input until \n doesn't occur.

The syntax for gets() in C is:

```
char* gets(char* s)
```

Here, s is the Pointer which point s to the array of characters where the input taken as a string will be stored and returned. This function will return the string in s when it finishes successfully.
Let us look at the previous example, but with using the gets() function to understand more clearly.

```c
#include <stdio.h>

int main()
{
    // array to store string taken as input
    char sentence[30];

    // take user input
    printf("Enter any sentence: ");

    // use the gets method
    gets(sentence);

    // printing the input value
    printf("You entered: %s.", sentence);

    return 0;
}
```

Output

```
Enter any sentence: I am happy today
You entered: I am happy today.
```

This time the code gave the complete string, *I am happy today* as output, including the spaces. This is because we used the gets() method.
So far, gets() seems to be a great method to take string input in C with spaces.
But gets() doesn't care about the size of the character array passed to it. This means, in the above example, if the user input would be more than 30 characters long, the gets() function would still try to store it in the sentence[] array, but as there is not much available space, this would lead to buffer overflow. Because of this limitation, using the gets() function is not preferable. We will explore other methods in the upcoming sections.

Note: The gets() method is no longer a standard function according to the C11 standard ISO/IEC 9899:2011. But it is still available in libraries and will continue to be available for compatibility reasons.

2. fgets()

To overcome the above-listed limitation, we can use fgets() function. The fgets() function is similar to gets(), but we can also specify a maximum size for the string which will be taken as a string input in C from the user. This means that it reads either the complete line as a string and stores it in the array if the size is less than what is specified as a parameter (say: sz), else it reads only sz characters provided by the user so that no buffer overflow occurs.

The syntax for fgets() in C is:

```
char* fgets(char* s, int  sz, FILE* stream)
```

Here, **s** is the Pointer which point s to the array of characters where the input taken as a string will be stored and returned. The **sz** variable specifies the maximum allowed size for the user input and **stream** is used to denote the stream from which the input is to be taken (usually stdin, but it can also be a file).

Let us look at an example to understand more clearly.

```
#include <stdio.h>

int main()
{
    // array to store string taken as input
    char sentence[20];

    // take user input
    printf("Enter any sentence: ");

    // use the fgets method specifying the size of the array as the max
limit
    fgets(sentence, 20, stdin);

    // printing the input value
    printf("You entered: %s.", sentence);

    return 0;
}
```

Output

```
Enter any sentence: I am going to the park today
You entered: I am going to the p.
```

Only 20 characters of the input were stored in the char array, 'sentence' as 20 was specified as the maximum size in fgets() and hence buffer overflow was avoided.

Comparision between gets() and fgets()

Although gets() and fgets() are similar in the sense that they both take a complete line as input, but they also have many dissimilarities.
While gets() can take string input in C only from the standard input stream, fgets() can take string input in C from both the standard input stream or from some file. Also, while the gets() method converts the \n character to \0 to make it a null-terminated string, the fgets() method does not do so. Instead, it adds a \0 symbol after the \n character to achieve the same.

Using scanset %[ ] in scanf()

Scanset is denoted by %[]. It can be used inside the scanf() function, and we can enclose individual characters or a range of characters within the scanset expression. Using this scanf() will only process these characters which have been specified inside the square brackets. We can also include multiple characters by adding commas in between the characters.

There are two ways by which we can use scanset to take input of a string in C with spaces.

1. {%[^\n]%*c} inside scanf

We can use the expression {%[^\n]%*c} inside scanf() to take the complete line including spaces as a string input in C.
Let us look at this expression in detail. In this case, we have ^\n inside the scanset brackets. The circumflex symbol ^ is used in this case. The use of this symbol is that it directs the specifier to stop reading once the character specified after it occurs once in the input. In this case, we have the \n newline character. Hence all the characters are taken as input until a new line occurs. As the newline character will also be taken as input, we discard it by further including %*c, so that only the user input is stored in the string.
The complete syntax for this method is:

```
scanf("%[^\n]%*c", s);
```

Here, **s** is the Pointer which point s to the array of characters where the input taken as a string will be stored.

Let us look at an example to understand more clearly.

```
#include <stdio.h>

int main()
{
    // array to store string taken as input
    char sentence[20];

    // take user input
    printf("Enter any sentence: ");

    // use scanf to get input
    scanf("%[^\n]%*c",sentence);

    // printing the input value
    printf("You entered: %s.", sentence);

    return 0;
}
```

Output

```
Enter any sentence: I am fine
You entered: I am fine.
```

2. %[^\n]s inside scanf

We can also use the expression %[^\n]s inside scanf() to take the complete line including spaces as a string input in C.
Let us look at this expression in detail. The characters [] denote the scanset character. In this case, we have ^\n inside the brackets, the circumflex symbol at the start will make sure all the characters are taken as input until the newline character \n is encountered. The expression [^\n] is included between the % and the s characters as %s is used as an access specifier for taking strings as input in C.

The complete syntax for this method is:

scanf("%[^\n]s", s);
Here, **s** is the Pointer which point s to the array of characters where the input taken as a string in C will be stored.

Let us look at an example to understand more clearly.

```c
#include <stdio.h>

int main()
{
    // array to store string taken as input
    char sentence[20];

    // take user input
    printf("Enter any sentence: ");

    // use scanf to take input
    scanf("%[^\n]s", sentence);

    // printing the input value
    printf("You entered: %s.", sentence);

    return 0;
}
```

Output

```
Enter any sentence: It is a wonderful day
You entered: It is a wonderful day.
```

In the next sections, we will take a look at how we can string output in C.

How to Output a String Without Spaces in C?

1. using printf()

If we want to do a string output in C stored in memory and we want to output it as it is, then we can use the printf() function. This function, like scanf() uses the access specifier %s to output strings.

The complete syntax for this method is:

```
printf("%s", char *s);
```

Here, **s** is the Pointer which point s to the array of characters which contains the string which we need to output.

Let us look at an example to understand the printf() function in C.

```
#include <stdio.h>

int main()
{
    // array that stores the string which we need to output
    char sentence[20];

    // take user input
    printf("Enter any sentence: ");

    // use fgets to take the input
    fgets(sentence, 20, stdin);

    // printing the input value using printf
    printf("You entered: %s.", sentence);

    return 0;
}
```

Output

```
Enter any sentence: It is a sunny day
You entered: It is a sunny day.
```

This was the case when there was only one string and we had to output it as it is. But, if we want to print two strings and don't want space in between, we can use the fputs() function.

2. using fputs()

The fputs() function can be used to output two strings in C without space in between as it does not transfer the control to a new line so that even if another string is print ed after it, both of them will be on the same line.

The complete syntax for this method is:

```
int  fputs(const char* strptr, FILE* stream);
```

Here, **s** is the Pointer which point s to the array of characters which contains the string which we need to output.
Let us look at an example to understand this better.

```
#include <stdio.h>

int main()
{
    // array that stores first name of the student
    char f name[20];
```

```
    // array that stores last name of the student
    char l_name[20];

    // take user input
    printf("Enter your first name: \n");

    // use scanf() to get input
    scanf("%s", &f_name);

    // take user input
    printf("Enter your last name: \n");

    // use scanf() to get input
    scanf("%s", &l_name);

    // printing the input value using fputs()
    fputs(f_name, stdout);
    fputs(l_name, stdout);

    return 0;
}
```

Output

```
Enter your first name: Edward
Enter your last name: Franklin
EdwardFranklin
```

Both the strings have been print ed in the same line, without any space as we used the fputs() function, which doesn't transfer the control to the next line.

How to Output a String With Spaces in C?

We saw how we can output a string in C without spaces, but what if we want to output a string in C with spaces? We can use the following two methods:

1. using printf() with \n

We saw the printf() function which we used to output a string in C as it is to the user. But, if we wanted to output two strings with a space in between them or conversely output two strings in different lines, we could do this also by using the printf() function.

To output two strings in two separate lines in C, we will have to include the newline character \n in the printf() function. The newline character will make sure that the next time anything is print ed, it is print ed in the next line. The complete syntax for this method is:

```
printf("%s \n", char *s);
printf("%s \n", char *s1)
```

Here, **s** and **s1** are the Pointers which point to the array of characters which contain the strings that we need to output. Because of the \n newline character, **s1** string will be output to the user in the next line.

To output two strings with a space in between, but in the same line, we just need to include that space in the printf() function. The complete syntax for this method is:

```
printf("%s %s", char *s, char *s1);
```

Here, **s** and **s1** are the Pointers which point to the array of characters which contain the strings that we need to output.

Let us look at an example to understand this better.

```
#include <stdio.h>

int main()
{
    // char array that stores first name of the student
    char f_name[20];

    // char array that stores last name of the student
    char l_name[20];

    // take user input
    printf("Enter your first name: \n");

    // use scanf() to get input
    scanf("%s", &f_name);

    // take user input
    printf("Enter your last name: \n");

    // use scanf() to get input
    scanf("%s", &l_name);

    // printing the input value in separate lines using printf()
    printf("%s \n", f_name);
    printf("%s \n", l_name);

    // printing the input value in the same line with a space using
printf()
    printf("%s %s", f_name, l_name);

    return 0;

}
```

Output

```
Enter your first name: Edward
Enter your last name: Franklin
Edward
Franklin
Edward Franklin
```

2. using puts()

Another function which we can use to output a string in C is the puts() function.

The thing to note however is that after print ing the given string in the output, the puts() function transfers the control to the next line. So any other string that we print after the execution of the puts()line will be print ed in the next line by default and not in the same line.

The complete syntax for this method is:

```
int  puts(const char* strptr);
```

Here, **strptr** is the Pointer which point s to the array of characters which contains the string which we need to output.

Let us look at an example to understand this better.

```
#include <stdio.h>

int main()
{
    // array that stores first name of the student
    char f_name[20];

    // array that stores last name of the student
    char l_name[20];

    // take user input
    printf("Enter your first name: \n");

    // use fgets to get input
    fgets(f_name,20,stdin);

    // take user input
    printf("Enter your last name: \n");

    // use fgets to get input
    fgets(l_name,20,stdin);

    // printing the input value using puts()
    puts(f_name);
    puts(l_name);

    return 0;

}
```

Output

```
Enter your first name: Edward
Enter your last name: Franklin
Edward
Franklin
```

As you can see in the code, we never specified that we need to leave a line after print ing the first name, but the code still did, because of using the puts() function, which by default transfers control to the next line after print ing.

Comparision between puts() and fputs()

The puts() and fputs() functions both are used to output strings to the user. Let us also look at the differences in their working.
While the puts() method converts the null termination character \0 to the newline character \n, the fgets() method does not do so. Hence, in the case of puts(), the control is transferred to a new line in the output, but in case of fputs(), it remains in the same line.

Examples
Let us look at some examples to revise the methods described above.

Example 1:
Program that takes student's name (with spaces) and city (without spaces) as input and print s them.

```c
#include <stdio.h>

int main()
{
    // char array that stores complete name of the student
    char name[50];

    // take user input
    printf("Enter your full name: \n");

    // use gets() to take input with spaces
    gets(name);

    // char array that stores city of the student
    char city_name[30];

    // take user input
    printf("Enter your city name: \n");

    // use scanf to take input without spaces
    scanf("%s", city_name);

    // print name with space using printf
    printf("Your name is %s \n", name);

    printf("You live in ");

    // print city using fputs
    fputs(city_name, stdout);

    return 0;
}
```

Output

```
Enter your complete name: Lily Collins
Enter your city name: Houston
Your name is Lily Collins
You live in Houston
```

In the above code, the complete name of a person may have spaces, hence we take it as input by using gets() and output it using prinf() with \n, whereas the city name won't have any spaces, and so we take it as input using scanf() and output it using fputs().

Example 2:
Saving a security question and answer for a user

```c
#include <stdio.h>

int main()
{
    // char array that stores name of the user
    char name[50];

    // take user input
    printf("Enter your name: \n");

    // use scanf to take input without spaces
    scanf("%s", name);

    // char array that stores security question
    char question[40];

    // take user input
    printf("Enter an appropriate security question: \n");

    // use gets to take input with spaces
    gets(question);

    // char array that stores answer of the question
    char answer[30];

    // take user input
    printf("Enter the answer of your question: \n");

    // use scanf to take input without spaces
    scanf("%s", answer);

    // print name and question using printf
    printf("Welcome %s \n", name);

    printf("The question is: %s \n", question);

    // print answer for the question using printf
    printf("The answer stored is: %s \n", answer);

    printf("Success!");


    return 0;

}
```

Output

```
Enter your name: Lily
Enter an appropriate security question: What was my first pet's name?
Enter the answer of your question: Bruno
Welcome Lily
```

```
The question is: What was my first pet's name?
The answer stored: is Bruno
Success!
```

This code uses scanf(), printf() and gets() functions to take string input in C and string output in C with and without spaces.

Introduction to Pointers:

Pointers in C are used to store the address of variables or a memory location. This variable can be of any data type i.e, int , char, function, array, or any other Pointer. Pointers are one of the core concepts of C programming language that provides low-level memory access and facilitates dynamic memory allocation.

What is a Pointer in C?

A Pointer is a derived data type in C that can store the address of other variables or a memory. We can access and manipulate the data stored in that memory location using Pointers.

Syntax of C Pointers

datatype * *Pointer_name*;

The above syntax is the generic syntax of C Pointers. The actual syntax depends on the type of data the Pointer is point ing to.

How to Use Pointers?

To use Pointers in C, we must understand below two operators:

1. Addressof Operator

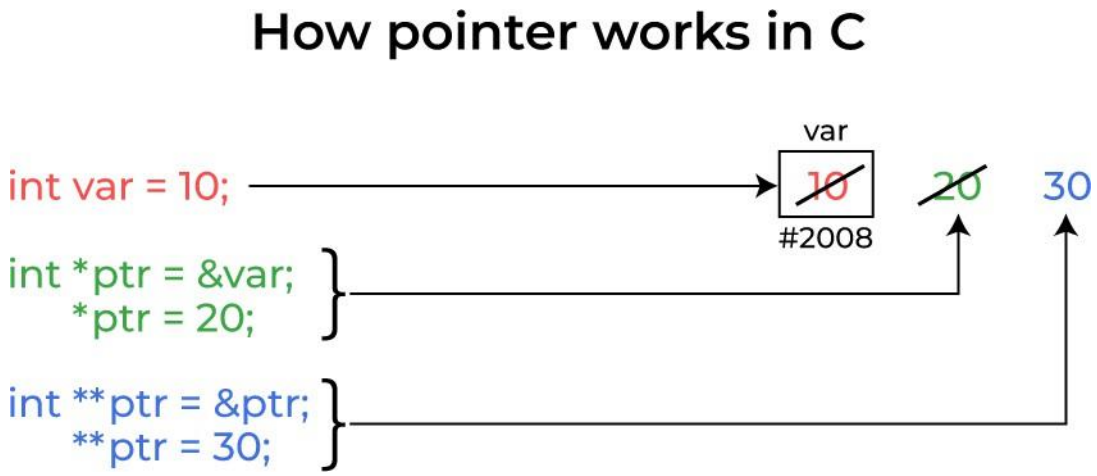
The addressof operator (&) is a unary operator that returns the address of its operand. Its operand can be a variable, function, array, structure, etc.

Syntax of Address of Operator

&*variable_name*;

2. Dereferencing Operator

The dereference operator (*), also known as the indirection operator is a unary operator. It is used in Pointer declaration and dereferencing.



C Pointer Declaration

In Pointer declaration, we only declare the Pointer but do not initialize it. To declare a Pointer, we use the * dereference operator before its name.

data_type * Pointer_name;

The Pointer declared here will point to some random memory address as it is not initialized. Such Pointers are also called wild Pointers that we will study later in this article.

C Pointer Initialization

When we assign some value to the Pointer, it is called Pointer Initialization in C. There are two ways in which we can initialize a Pointer in C of which the first one is:

Method 1: C Pointer Definition

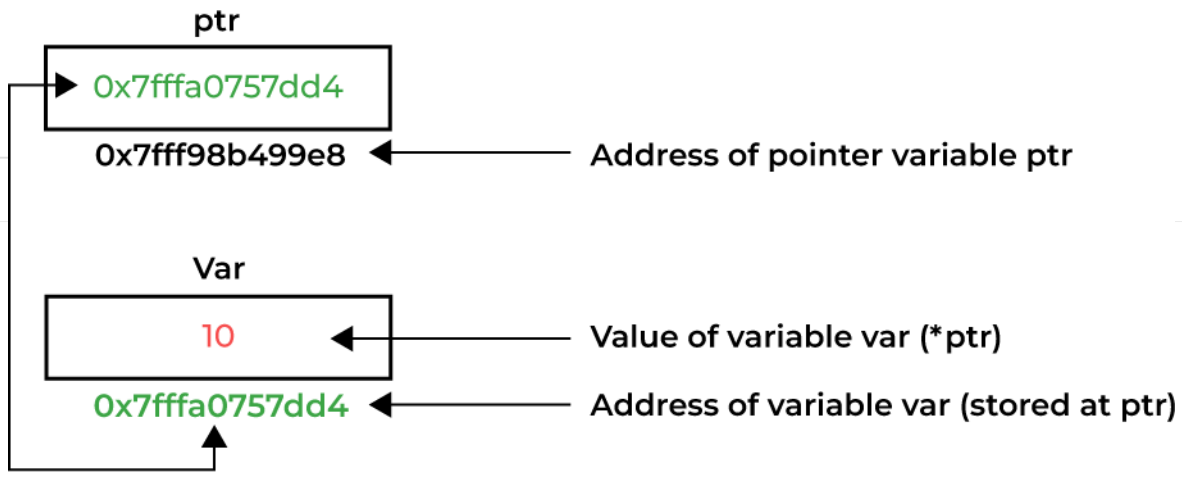
datatype * Pointer_name = *address*;
The above method is called Pointer Definition as the Pointer is declared and initialized at the same time.

Method 2: Initialization After Declaration

The second method of Pointer initialization in C the assigning some address after the declaration.
datatype * Pointer_name; Pointer_name = *addresss*;

Dereferencing a C Pointer

Dereferencing is the process of accessing the value stored in the memory address specified in the Pointer. We use dereferencing operator for that purpose.



Dereferencing a Pointer in C

Example: C Program to demonstrate how to use Pointers in C.

- C

```
// C program to illustrate Pointers #include
<stdio.h>
void geeks()
{
        int var = 20;

        // declare Pointer variable
```

```
    int * ptr;

        // note that data type of ptr and var must be same ptr = &var;
        // assign the address of a variable to a Pointer printf("Value
        at ptr = %p \n", ptr); printf("Value at var = %d \n", var);
        printf("Value at *ptr = %d \n", *ptr);
}
// Driver program
int main()
{
        geeks();
        return0;
}
```

Output

Value at ptr = 0x7ffd15b5deec Value at
var = 20
Value at *ptr = 20

Types of Pointers

Pointers can be classified int o many different types based on the parameter on which we are defining their types. If we consider the type of variable stored

in the memory location point ed by the Pointer, then the Pointers can be classified int o the following types:

1. Integer Pointers

As the name suggests, these are the Pointers that point to the integer values.

Syntax of Integer Pointers

int **Pointer_name*;

These Pointers are pronounced as Pointer to Integer.

Similarly, a Pointer can point to any primitive data type. The syntax will change accordingly. It can point also point to derived data types such as arrays and user-defined data types such as structures.

2. Array Pointer

Pointers and Array are closely related to each other. Even the array name is the Pointer to its first element. They are also known as Pointer to Arrays. We can create a Pointer to an array using the given syntax.

Syntax of Array Pointers

char **Pointer_name* = &*array_name*;

Pointer to Arrays exhibits some int eresting properties which we discussed later in this article.

3. Structure Pointer

The Pointer point ing to the structure type is called Structure Pointer or Pointer to Structure. It can be declared in the same way as we declare the other primitive data types.

Syntax of Structure Pointer

struct *struct_name* **Pointer_name*;

4. Function Pointers

Function Pointers point to the functions. They are different from the rest of the Pointers in the sense that instead of point ing to the data, they point to the code. Let's consider a

function prototype — **int func (int , char)**, the function Pointer for this function will be
Syntax of Function Pointer
int (*Pointer_name)(int , int);
Keep in mind that the syntax of the function Pointers changes according to the function prototype.

5. Double Pointers

In C language, we can define a Pointer that stores the memory address of another Pointer. Such Pointers are called double-Pointers or Pointers-to- Pointer. Instead of point ing to a data value, they point to another Pointer.
Syntax of Double Pointer in C
datatype ** Pointer_name;
Dereferencing in Double Pointer
*Pointer_name; // get the address stored in the inner level Pointer
**Pointer_name; // get the value point ed by inner level Pointer
Note: *In C, we can create multi-level Pointers with any number of levels such as – ***ptr3, ****ptr4, ******ptr5 and so on.*

6. NULL Pointer

The Null Pointers are those Pointers that do not point to any memory location. They can be created by assigning a NULL value to the Pointer. A Pointer of any type can be assigned the NULL value.
Syntax of NULL Pointer in C
*data_type *Pointer_name* = NULL; or
Pointer_name = NULL
It is said to be good practice to assign NULL to the Pointers currently not in use.

7. Void Pointer

The Void Pointers in C are the Pointers of type void. It means that they do not have any associated data type. They are also called **generic Pointers** as they can point to any type and can be typecasted to any type.
Syntax of Void Pointer
void * *Pointer_name*;
One of the main properties of void Pointers is that they cannot be dereferenced.

8. Wild Pointers

The Wild Pointers are Pointers that have not been initialized with something yet. These types of C-Pointers can cause problems in our programs and can eventually cause them to crash.
Example of Wild Pointers
int *ptr; char
*str;

9. Constant Pointers

In constant Pointers, the memory address stored inside the Pointer is constant and cannot be modified once it is defined. It will always point to the same memory address.

Syntax of Constant Pointer

const *data_type * Pointer_name*;

10. Pointer to Constant

The Pointers point ing to a constant value that cannot be modified are called Pointers to a constant. Here we can only access the data point ed by the Pointer, but cannot modify it. Although, we can change the address stored in the Pointer to constant.

Syntax to Pointer to Constant data_type

* const *Pointer_name*; Other Types of

Pointers in C:

There are also the following types of Pointers available to use in C apart from those specified above:

- **Far Pointer:** A far Pointer is typically 32-bit that can access memory outside the current segment.
- **Dangling Pointer:** A Pointer point ing to a memory location that has been deleted (or freed) is called a dangling Pointer.
- **Huge Pointer:** A huge Pointer is 32-bit long containing segment address and offset address.
- **Complex Pointer:** Pointers with multiple levels of indirection.
- **Near Pointer:** Near Pointer is used to store 16-bit addresses means within the current segment on a 16-bit machine.
- **Normalized Pointer:** It is a 32-bit Pointer, which has as much of its value in the segment register as possible.
- **File Pointer:** The Pointer to a FILE data type is called a stream Pointer or a file Pointer.

Size of Pointers in C

The size of the Pointers in C is equal for every Pointer type. The size of the Pointer does not depend on the type it is point ing to. It only depends on the operating system and CPU architecture. The size of Pointers in C is

- **8 bytes** for a **64-bit System**
- **4 bytes** for a **32-bit System**

The reason for the same size is that the Pointers store the memory addresses, no matter what type they are. As the space required to store the addresses of the different memory locations is the same, the memory required by one Pointer type will be equal to the memory required by other Pointer types.

How to find the size of Pointers in C?

We can find the size of Pointers using the **sizeof** operator as shown in the following program:

Example: C Program to find the size of different Pointer types.

- C

```
// C Program to find the size of different Pointers types #include <stdio.h>
// dummy structure
struct str
{
};
```

// dummy function

```
void func(int a, int b)
{};
int main()
{
    // dummy variables definitions int a =
    10;
    charc = 'G'; struct
    str x;
// Pointer definitions of different types int * ptr_int  = &a;
    char* ptr_char = &c; structstr*
    ptr_str = &x;
    void(*ptr_func)(int , int ) = &func; void* ptr_vn
    = NULL;
    // print ing sizes
    printf("Size of Integer Pointer  \t:\t%d bytes\n", sizeof(ptr_int ));
    printf("Size of Character Pointer\t:\t%d bytes\n", sizeof(ptr_char));
    printf("Size of Structure Pointer\t:\t%d bytes\n", sizeof(ptr_str));
    printf("Size of Function Pointer\t:\t%d bytes\n", izeof(ptr_func));
    printf("Size of NULL Void Pointer\t:\t%d bytes", sizeof(ptr_vn));
    return0;

}
```

Output

```
Size of Integer Pointer          :    8 bytes
Size of Character Pointer        :    8 bytes
Size of Structure Pointer        :    8 bytes
Size of Function Pointer         :    8 bytes
Size of NULL Void Pointer        :    8 bytes
```

As we can see, no matter what the type of Pointer it is, the size of each and every Pointer is the same.

Now, one may wonder that if the size of all the Pointers is the same, then why do we need to declare the Pointer type in the declaration? The type declaration is needed in the Pointer for dereferencing and Pointer arithmetic purposes.

Pointer Arithmetic

Only a limited set of operations can be performed on Pointers. The Pointer Arithmetic refers to the legal or valid operations that can be performed on a Pointer. It is slightly different from the ones that we generally use for mathematical calculations. The operations are:

- Increment in a Pointer
- Decrement in a Pointer
- Addition of integer to a Pointer
- Subtraction of integer to a Pointer
- Subtracting two Pointers of the same type
- Comparison of Pointers of the same type.
- Assignment of Pointers of the same type.

- C

```
// C program to illustrate Pointer Arithmetic
#include <stdio.h>
int main()
{
        // Declare an array
        int v[3] = { 10, 100, 200 };
        // Declare Pointer variable int * ptr;
        // Assign the address of v[0] to ptr ptr = v;
        for(int i = 0; i < 3; i++)
        {
                // print  value at address which is stored in ptr printf("Value of *ptr =
                %d\n", *ptr);
                // print  value of ptr
                printf("Value of ptr = %p\n\n", ptr);
                // Increment Pointer ptr by 1 ptr++;
        }
        return0;
}
```

Output

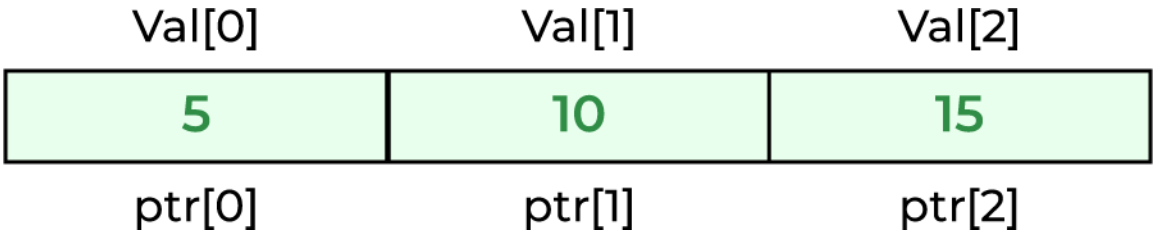
Value of *ptr = 10
Value of ptr = 0x7ffe8ba7ec50
Value of *ptr = 100
Value of ptr = 0x7ffe8ba7ec54
Value of *ptr = 200
Value of ptr = 0x7ffe8ba7ec58

C Pointers and Arrays Relation

In C programming language, Pointers and arrays are closely related. An array name acts like a Pointer constant. The value of this Pointer constant is the address of the first element. For example, if we have an array named val then **val** and **&val[0]** can be used int erchangeably.

If we assign this value to a non-constant Pointer to the array, then we can access the elements of the array using this Pointer.

Example 1: Accessing Array Elements using Pointer with Array Subscript



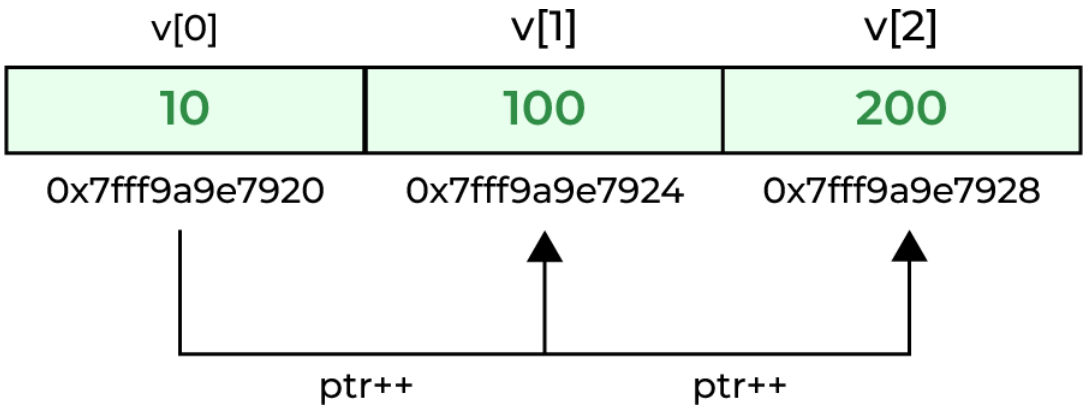
```
// C Program to access array elements using Pointer #include
<stdio.h>

void geeks()

{

        // Declare an array

        int val[3] = { 5, 10, 15 };

        // Declare Pointer variable int * ptr;

        // Assign address of val[0] to ptr.

        // We can use ptr=&val[0];(both are same) ptr =
        val;
        printf("Elements of the array are: ");
        printf("%d, %d, %d", ptr[0], ptr[1], ptr[2]);
        return;
}

// Driver program int
main()
{
        geeks();
        return0;
}
```

Output

Elements of the array are: 5 10 15

Not only that, as the array elements are stored continuously, we can Pointer arithmetic operations such as increment, decrement, addition, and subtraction of integers on Pointer to move between array elements.

Example 2: Accessing Array Elements using Pointer Arithmetic



```
// C Program to access array elements using Pointers #include
<stdio.h>
int main()
{
        // defining array
        int arr[5] = { 1, 2, 3, 4, 5 };
        // defining the Pointer to array int *
        ptr_arr = &arr;
        // traversing array using Pointer arithmetic for(int i = 0;
        i < 5; i++) {
                printf("%d ", *ptr_arr++);

        }
        return 0;
}
```

Output

1 2 3 4 5

This concept is not limited to the one-dimensional array, we can refer to a multidimensional array element perfectly fine using this concept.

Uses of Pointers

The C Pointer is a very powerful tool that is widely used in C programming to perform various useful operations. It finds its use in operations such as

1. Pass Arguments by Reference
2. Accessing Array Elements
3. Return Multiple Values from Function
4. Dynamic Memory Allocation
5. Implementing Data Structures
6. In System-Level Programming where memory addresses are useful.
7. In locating the exact value at some memory location.
8. To avoid compiler confusion for the same variable name.
9. To use in Control Tables.

Advantages of Pointers

- Pointers are used for dynamic memory allocation and deallocation.
- An Array or a structure can be accessed efficiently with Pointers
- Pointers are useful for accessing memory locations.
- Pointers are used to form complex data structures such as linked lists, graphs, trees, etc.
- Pointers reduce the length of the program and its execution time as well.

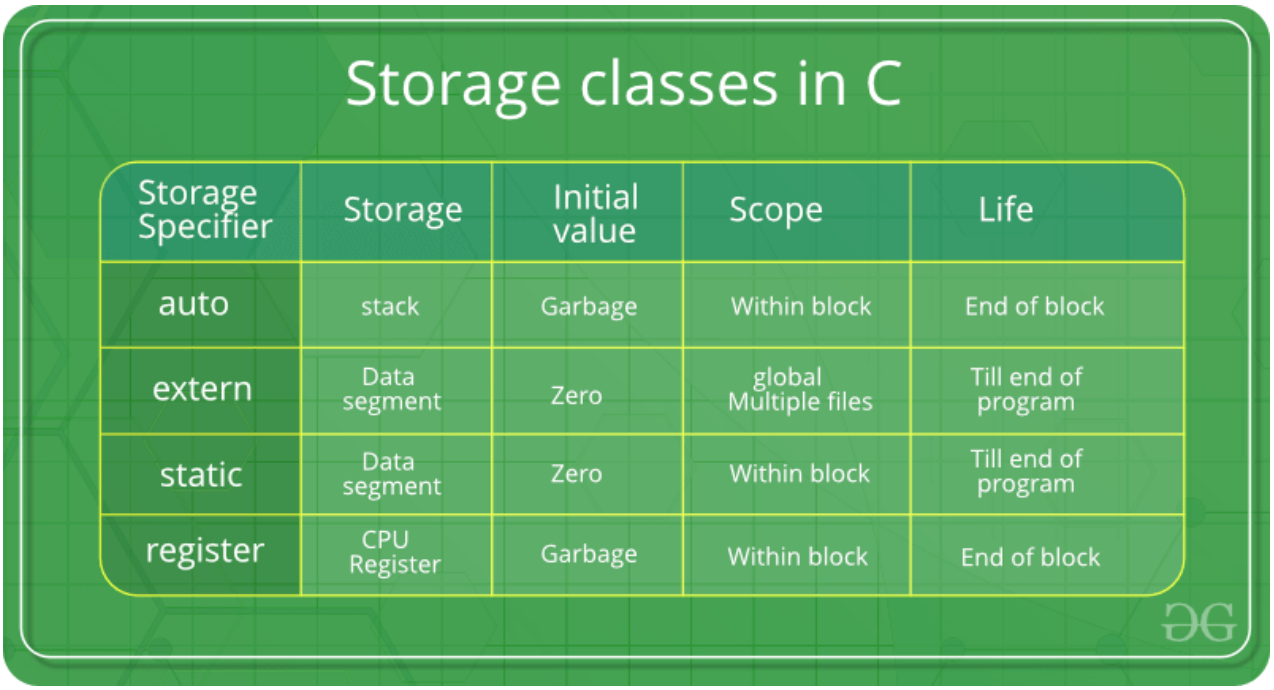
Disadvantages of Pointers

- Memory corruption can occur if an incorrect value is provided to Pointers.
- Pointers are a little bit complex to understand.
- Pointers are majorly responsible for memory leaks in C.
- Pointers are comparatively slower than variables in C.
- Uninitialized Pointers might cause a segmentation fault.

STORAGE CLASS IN C

Storage Classes are used to describe the features of a variable/function. These features basically include the scope, visibility and life-time which help us to trace the existence of a particular variable during the runtime of a program.

C language uses 4 storage classes, namely:



1. **auto**: This is the default storage class for all the variables declared inside a function or a block. Hence, the keyword auto is rarely used while writing programs in C language. Auto variables can be only accessed within the block/function they have been declared and not outside them (which defines their scope). Of course, these can be accessed within nested blocks within the parent block/function in which the auto variable was declared. However, they can be accessed outside their scope as well using the concept of Pointers given here by point ing to the very exact memory location where the variables reside. They are assigned a garbage value by default whenever they are declared.
2. **extern**: Extern storage class simply tells us that the variable is defined elsewhere and not within the same block where it is used. Basically, the value is assigned to it in a different block and this can be overwritten/changed in a different block as well. So an extern variable is nothing but a global variable initialized with a legal value where it is declared in order to be used elsewhere. It can be accessed within any function/block. Also, a normal global variable can be made extern as well by placing the 'extern' keyword before its declaration/definition in any function/block. This basically signifies that we are not initializing a new variable but instead we are using/accessing the global variable only. The main purpose of using extern variables is that they can be accessed between two different files which are part of a large program. For more information on how extern variables work, have a look at this link.
3. **static**: This storage class is used to declare static variables which are popularly used while writing programs in C language. Static variables have the property of preserving their value even after they are out of their scope! Hence, static variables preserve the value of their last use in their scope. So we can say that they are initialized only once and exist till the termination of the program. Thus, no new memory is allocated because they are not re-declared. Their scope is local to the function to which they were defined. Global static variables can be accessed anywhere in the program. By default, they are assigned the value 0 by the compiler.

4. **register**: This storage class declares register variables that have the same functionality as that of the auto variables. The only difference is that the compiler tries to store these variables in the register of the microprocessor if a free register is available. This makes the use of register variables to be much faster than that of the variables stored in the memory during the runtime of the program. If a free registration is not available, these are then stored in the memory only. Usually few variables which are to be accessed very frequently in a program are declared with the register keyword which improves the running time of the program. An important and int eresting point to be noted here is that we cannot obtain the address of a register variable using Pointers.

To specify the storage class for a variable, the following syntax is to be followed:
Syntax:
storage_class var_data_type var_name;
Functions follow the same syntax as given above for variables. Have a look at the following C example for further clarification:

C

```
// A C program to demonstrate different storage

// classes
#include <stdio.h>
// declaring the variable which is to be made extern
// an initial value can also be initialized to x int x;
void autoStorageClass()
{
        printf("\nDemonstrating auto class\n\n");
        // declaring an auto variable (simply
        // writing "int  a=32;" works as well) auto int a = 32;
        // print ing the auto variable 'a' printf("Value of the variable 'a'"
                    " declared as auto: %d\n", a);
        printf("    ----------------------------------------------");

}
void registerStorageClass()
{
    printf("\nDemonstrating register class\n\n");
        // declaring a register variable register char b = 'G';
        // print ing the register variable 'b'
        printf("Value of the variable 'b' declared as register: %d\n", b);
        printf("    ----------------------------------------------");
}
void externStorageClass()
{
printf("\nDemonstrating extern class\n\n");
        // telling the compiler that the variable
        // x is an extern variable and has been
        // defined elsewhere (above the main
        // function) extern
        int x;
        // print ing the extern variables 'x'
         printf("Value of the variable 'x' declared as extern:
        %d\n", x);
```

```
        // value of extern variable x modified x = 2;
        // print ing the modified values of
        // extern variables 'x'
        printf("Modified value of the variable 'x'" " declared as extern: %d\n", x);
        printf("    ----------------------------------------------");

}


void staticStorageClass()
{
        int i = 0;
        printf("\nDemonstrating static class\n\n");
        // using a static variable 'y'
        printf("Declaring 'y' as static inside the loop.\n" "But this declaration will occur only"
                        " once as 'y' is static.\n"
                        "If not, then every time the value of 'y' " "will be the declared value 5"
                        " as in the case of variable 'p'\n");
        printf("\nLoop started:\n");
    for(i = 1; i < 5; i++) {

                // Declaring the static variable 'y' staticint y = 5;
                // Declare a non-static variable 'p' int p = 10;
                // Incrementing the value of y and p by 1 y++;
                p++;
                // print ing value of y at each iteration
                printf("\nThe value of 'y', ""declared as static, in %d " "iteration is %d\n",i, y);
                // print ing value of p at each iteration printf("The value of non-static variable 'p', "
                                "in %d iteration is %d\n", i, p);

        }
        printf("\nLoop ended:\n");
        printf("    ----------------------------------------------");
}
int main()
{
        printf("A program to demonstrate Storage Classes in C\n\n");

        // To demonstrate auto Storage Class autoStorageClass();
        // To demonstrate register Storage Class registerStorageClass();
        // To demonstrate extern Storage Class externStorageClass();
        // To demonstrate static Storage Class staticStorageClass();
        // exiting
        printf("\n\nStorage Classes demonstrated");
        return0;
}
```

Output

...ster: 71

Demonstrating extern class

Value of the variable 'x' declared as extern: 0

Modified value of the variable 'x' declared as extern: 2

Demonstrating static class

Declaring 'y' as static inside the loop.

But this declaration will occur only once as 'y' is static.
If not, then every time the value of 'y' will be the declared value
5 as in the case of variable 'p'

Loop started:

The value of 'y', declared as static, in 1 iteration is 6
The value of non-static variable 'p', in 1 iteration is 11

The value of 'y', declared as static, in 2 iteration is 7
The value of non-static variable 'p', in 2 iteration is 11

The value of 'y', declared as static, in 3 iteration is 8
The value of non-static variable 'p', in 3 iteration is 11

The value of 'y', declared as static, in 4 iteration is 9
The value of non-static variable 'p', in 4 iteration is 11

Loop ended:

Storage Classes demonstrated

Algorithm Development in C:

Algorithm is a step – by – step procedure which is helpful in solving a problem. If, it is written in English like sentences then, it is called as ‘PSEUDO CODE’.

Properties of an Algorithm

An algorithm must possess the following five properties −

- Input
- Output
- Finiteness
- Definiteness
- Effectiveness

An algorithm is a sequence of instructions that are carried out in a predetermined sequence in order to solve a problem or complete a work. A function is a block of code that can be called and executed from other parts of the program.

- A set of instructions for resolving an issue or carrying out a certain activity. In computer science, algorithms are used for a wide range of operations, from fundamental math to int ricate data processing.
- One of the common algorithms used in C is the sorting algorithm. A sorting algorithm arranges a collection of items in a certain order, such as numerically or alphabetically.
- There are many sorting algorithms, each with advantages and disadvantages. The most common sorting algorithms in C are quicksort, merge, and sort.

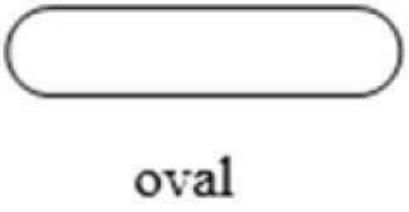
Example

Algorithm for finding the average of three numbers is as follows −

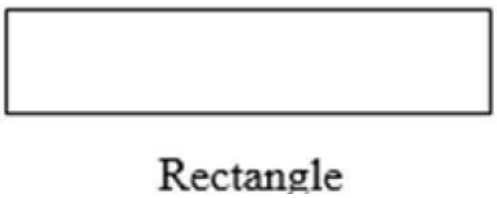
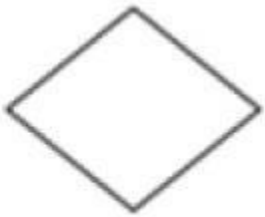
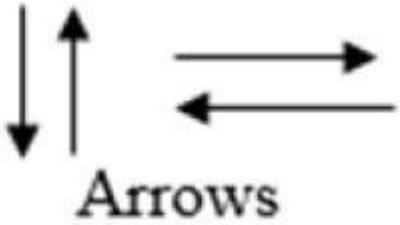
- Start
- Read 3 numbers a,b,c
- Compute sum = a+b+c
- Compute average = sum/3
- Print average value
- Stop

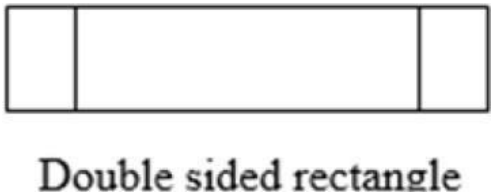
FLOW CHART

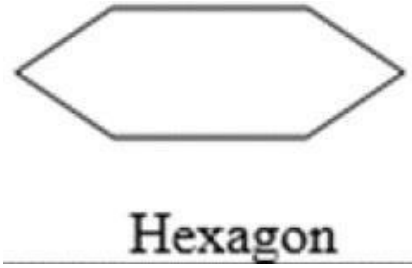
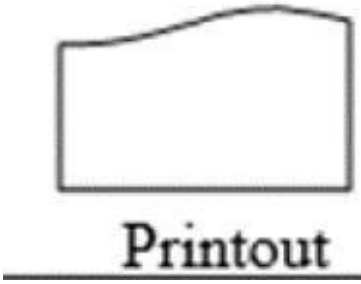
Diagrammatic representation of an algorithm is called flow chart. Symbols used in flowchart are mentioned below —

Name	Symbol	Purpose
Terminal	 Oval	start/stop/begin/end

Name	Symbol	Purpose
Input/output	 Parallelogram	Input/output of data

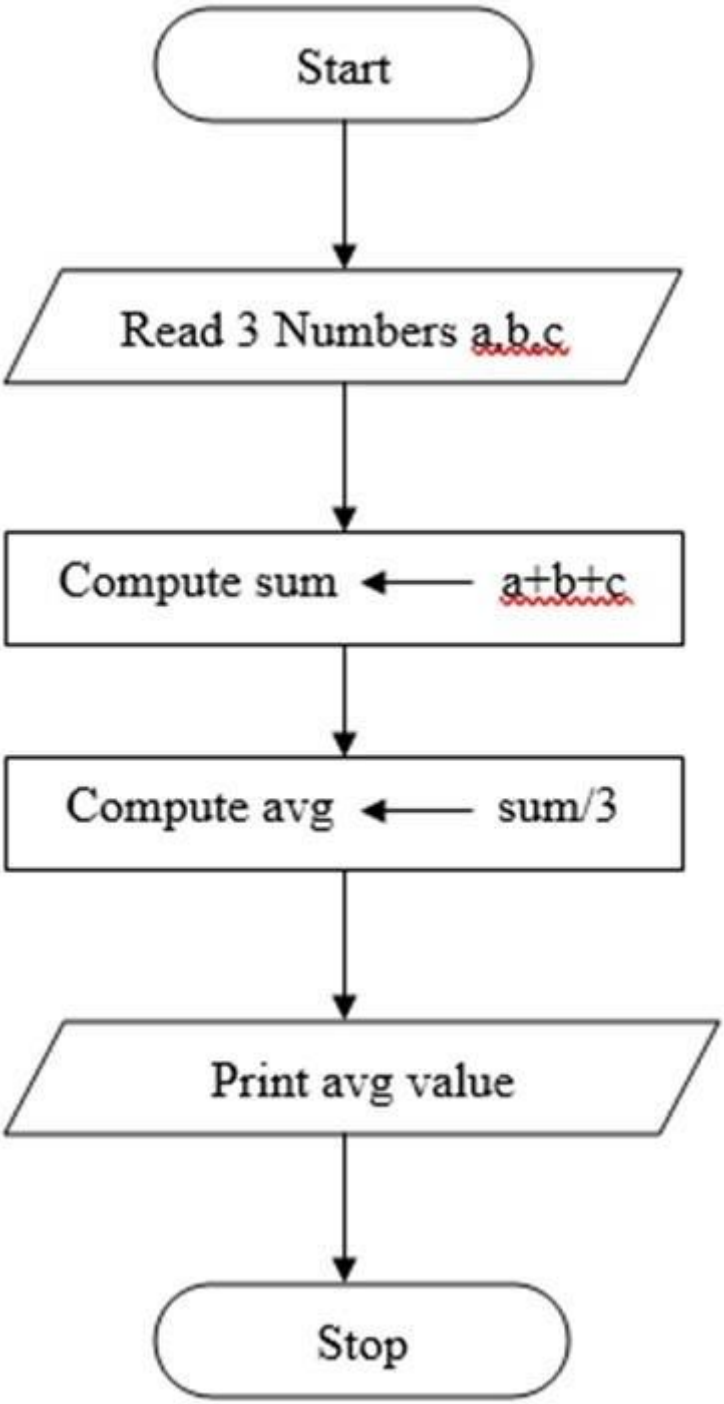
Process	 Rectangle	Any processing to be performaed can be represented
Decision box	 Diamon	Decision operation that determine which of the alternative paths to be followed
Connector	 Circle	Used to connect different parts of flowchart
Flow	 Arrows	Join 2 symbols and also represents flow of execution

Name	Symbol	Purpose
Pre defined process	 Double Sided Rectangle	Module (or) subroutines specified else where

Page connecto r	 Pentagon	Used to connect flowchart in 2 different pages
For loop symbol	 Hexagon	shows initialization, condition and incrementation of loop variable
Document	 Print out	Shows the data that is ready for print out

Example

Given below is the flowchart for finding an average of three numbers −



Features of the algorithm

It defines several important features of the algorithm, including:

- **Inputs**: Algorithms must receive inputs that can be represented as values or data.
- **Output**: The algorithm should produce some output. It can be a consequence of a problem or a solution designed to solve it.

- **Clarity**: Algorithms must be precisely defined, using unambiguous instructions that a computer or other system can follow unambiguously.
- **Finiteness**: The algorithm requires a limited steps. It means that it should be exited after executing a certain number of commands.
- **Validity**: The algorithm must be valid. In other words, it should be able to produce a solution to the problem that the algorithm is designed to solve in a reasonable amount of time.
- **Effectiveness:** An algorithm must be effective, meaning that it must be able to produce a solution to the problem it is designed to solve in a reasonable amount of time.
- **Generality:** An algorithm must be general, meaning that it can be applied to a wide range of problems rather than being specific to a single problem.

Algorithm Analysis

Algorithmic analysis is the process of evaluating algorithm performance in terms of efficiency, complexity and other criteria.Typically, this is done to evaluate many algorithms and select the optimal solution for a certain issue or a software.

Analysis of algorithms usually involves measuring their time and space complexity.

As with space complexity, which describes the amount of memory or disk space needed, time complexity describes how long an algorithm determines to perform a task.

There are different ways to analyze the time complexity of algorithms, such as Big O and Omega notation. The Omega symbol provides an upper bound for the algorithm's time complexity, while the Omega symbol provides a lower bound.

In addition to measuring time and space complexity, algorithm analysis also includes other criteria such as stability, parallelism, and scalability.

1. **Stability**:- This refers to the ability of the algorithm to maint ain the relative order of the elements in the data set.
2. **Parallelization**:- This is referring to the capacity to execute operations in parallel across several processors.
3. **Scalability**:- On the other hand, it refers to the ability of an algorithm to handle large volumes of data and other inputs.

They include two types of analysis.

they are:-

1. Prior-analysis.
2. Posterior analysis.

Prior Analysis

Prior is a method of algorithm analysis that focuses on estimating the performance of an algorithm based on its mathematical properties without actually executing the algorithm.

This approach allows you to analyze the time and space complexity of algorithms and other metrics without the need to implement and run the algorithms.

Posterior analysis

Posterior analysis, on the other hand, is a method of algorithm analysis that actually executes the algorithm and measures its performance.

This approach provides more accurate and detailed information about the performance of the algorithm, but requires the implementation and execution of the algorithm.

Algorithm Complexity

Algorithmic complexity is a measure to measure the efficiency and performance of the algorithm. Algorithms are usually evaluated in terms of the time and space required to solve a problem or achieve a specific goal.

Two factors are used in the complexity of the algorithm. they
are:-

1. Time factor.
2. Space factor.

Time factor

- The amount of time an algorithm needs to do a task is referred to as time complexity.It is usually measured by the number of operations or steps an algorithm must perform to solve a problem.
- The time complexity of an algorithm is important because it determines how long it takes to execute and can have a significant impact on program and system performance.
- The time complexity of an algorithm can be expressed using Big O notation, a way of expressing an upper bound on the time complexity of an algorithm.
- An algorithm with time complexity O(n) means that the time required to run the algorithm is directly proportional to the size of the input data (n).
- Other common time complexities are O(n^2) quadratic complexity and O(log n) logarithmic complexity.

Space analysis

- On the other hand, space complexity refers to the amount of memory or storage space required to execute the algorithm.
- This is important because it determines the number of resources required to run algorithms that can affect the overall performance of your application or system.
- If the space complexity of the algorithm is O(n), it uses an amount of memory that grows linearly with the size of the input.
- If the algorithm has O(1) space complexity, it uses a fixed amount of memory regardless of the size of the input.

How to write an algorithm

1. First define the problem you want the algorithm to solve.

For example, suppose we want to write an algorithm to find the maximum value from a list of numbers.

2. Break the problem down int o smaller, manageable steps.

- Initialize the 'max' variable to the first value in the list.
- For each subsequent value in the list, compare with "max".
- If the value is greater than "max", set "max" to that value.
- Continue doing this until every value in the list has been compared.
- Returns the final "max" value.

Development of Efficient Program in C:

Efficient programming is programming in a manner that, when the program is executed, uses a low amount of overall resources pertaining to computer hardware. A program is designed by a human being, and different human beings may use different algorithms, or sequences of codes, to perform particular tasks. Some algorithms are more hefty and resource-int ensive while accomplishing the same task than another algorithm. Practicing to create a low file size and low resource algorithm results in an efficient program.

When we want to develop a program by using any programming language, we have to follow a sequence of steps. These steps are called phases in program development.

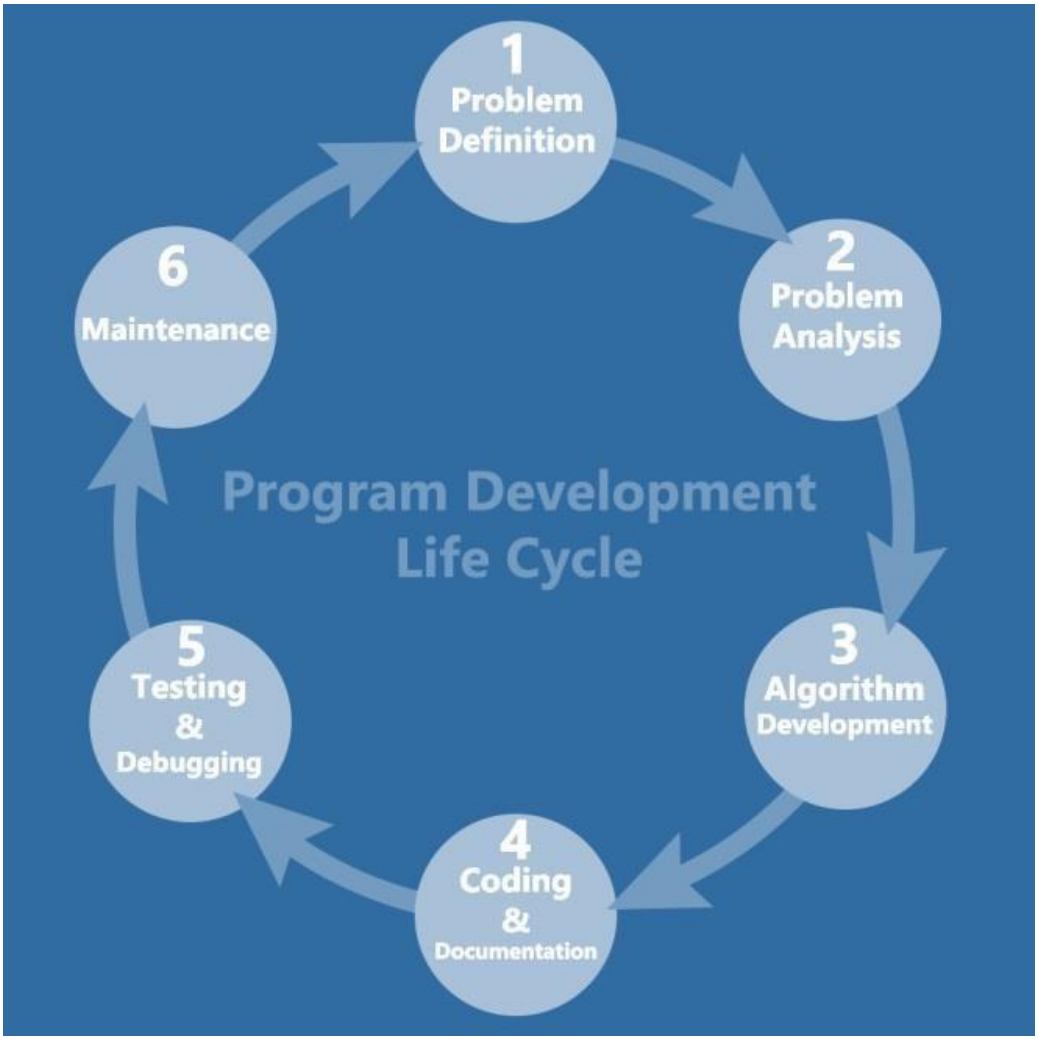
The program development life cycle is a set of steps or phases which are used to develop a program in any programming language.

Phases of program development

Program development life cycle contains 6 phases, which are as follows −

- Problem Definition.
- Problem Analysis.
- Algorithm Development.
- Coding & Documentation.
- Testing & Debugging.
- Maint enance.

These six phases are depicted in the diagram given below −



Problem Definition

Here, we define the problem statement and decide the boundaries of the problem.

In this phase, we need to understand what is the problem statement, what is our requirement and what is the output of the problem solution. All these are included in the first phase of program development life cycle.

Problem Analysis

Here, we determine the requirements like variables, functions, etc. to solve the problem. It means that we gather the required resources to solve the problem, which are defined in the problem definition phase. Here, we also determine the bounds of the solution.

Algorithm Development

Here, we develop a step-by-step procedure that is used to solve the problem by using the specification given in the previous phase. It is very important phase for the program development. We write the solution in step-by-step statements.

Coding & Documentation

Here, we use a programming language to write or implement the actual programming instructions for the steps defined in the previous phase. We construct the actual program in this phase. We write the program to solve the given problem by using the programming languages like C, C++, Java, etc.

Testing & Debugging

In this phase, we check whether the written code in the previous step is solving the specified problem or not. This means, we try to test the program whether it is solving the problem for various input data values or not. We also test if it is providing the desired output or not.

Maint enance

In this phase, we make the enhancements. Therefore, the solution is used by the end- user. If the user gets any problem or wants any enhancement, then we need to repeat all these phases from the starting, so that the encountered problem is solved or enhancement is added.